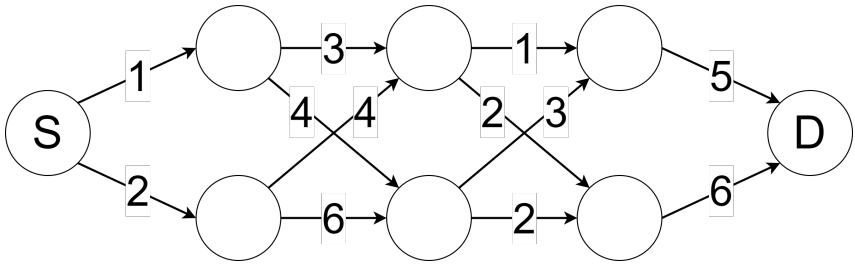
Dynamic Programming (DP) and Principle of Optimality

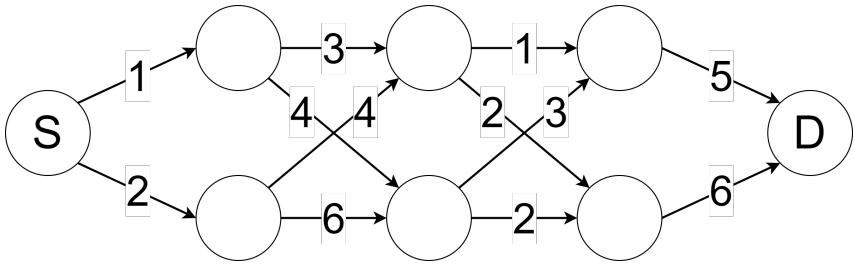
A Deterministic DP Example



Find the shortest path from S to D.

Exclusive search (forward search): number of possible paths $= 2 \times 2 \times 2 = 8$

A Deterministic DP Example

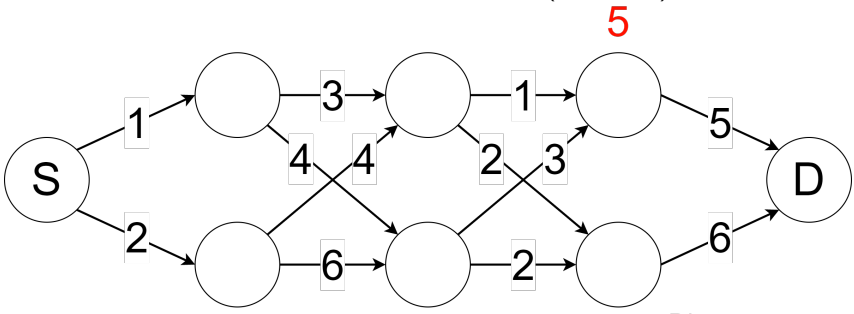


Find the shortest path from S to D.

DP (backward search): start from the destination

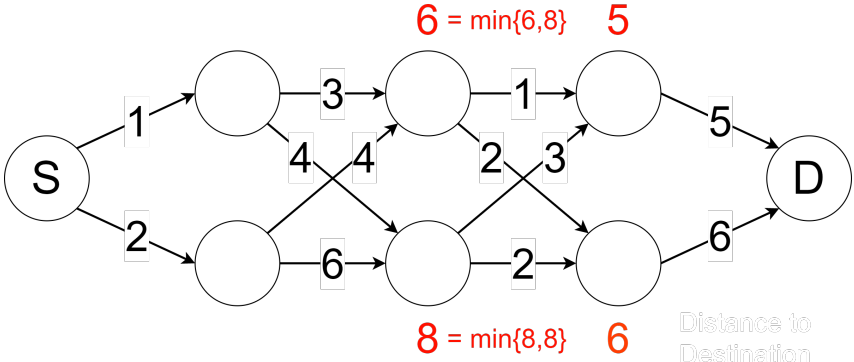
DP: backward search

- Calculate the shortest distance from a current node to destination D.
- Each calculation is a comparison of two choices (numbers)



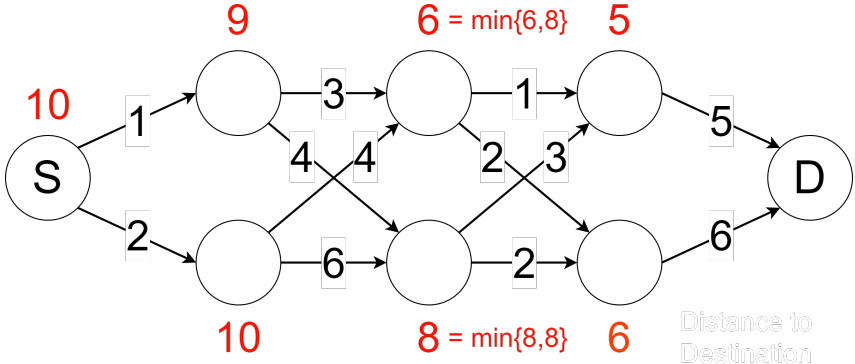
DP: backward search

- Calculate the shortest distance from a current node to destination D.
- Each calculation is a comparison of two choices (numbers)



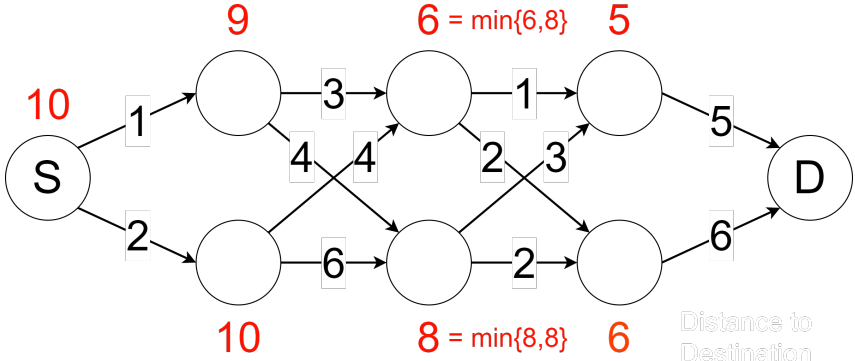
DP: backward search

- Calculate the shortest distance from a current node to destination D.
- Each calculation is a comparison of two choices (numbers)



DP: backward search

- Calculate the shortest distance from a current node to destination D.
- Each calculation is a comparison of two choices (numbers)



$1 + 2 + 2 = 5$ comparisons

Deterministic DP

The computational complexity of exclusive search versus DP.

Suppose the problem has T stages. Then

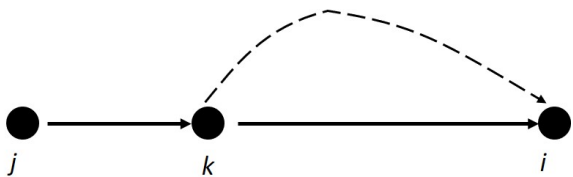
- Exclusive search: exponential in T
- Dynamic programming: linear in T

Principle of Optimality

A subsolution of an optimal solution is the optimal solution for the corresponding subproblem.

$$d(j,i) = \min_{k \in \mathcal{N}(j)} c(j,k) + d(k,i)$$

$\mathcal{N}(j)$ is set of outgoing neighbors of node j and $c(j,k)$ is the distance of edge (j,k).



Finite horizon deterministic DP

System equation:

$$x_{k+1} = f_k(x_k, u_k), \quad k = 0, 1, \ldots, N-1$$

x_k: system state at time k, u_k: control at time k

Finite horizon deterministic DP

System equation:

$$x_{k+1} = f_k(x_k, u_k), \quad k = 0, 1, \ldots, N-1$$

x_k: system state at time k, u_k: control at time k

In the shortest path problem,

x_k: on the top or bottom node at stage k

u_k: go to the top or bottom node at stage $k+1$

Finite horizon deterministic DP

For given initial state x_0, the cost over control sequence $(u_0, u_1, \ldots, u_{N-1})$ is defined to be

$$J(x_0; u_0, \ldots, u_{N-1}) = g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k)$$

Finite horizon deterministic DP

For given initial state x_0, the cost over control sequence $(u_0, u_1, \ldots, u_{N-1})$ is defined to be

$$J(x_0; u_0, \ldots, u_{N-1}) = g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k)$$

Optimal cost and optimal control:

$$J^*(x_0) = \min_{u_0, \ldots, u_{N-1}} J(x_0; u_0, \ldots, u_{N-1})$$

Finite horizon deterministic problem

Backward computation:

$$J_N^*(x_N) = g_N(x_N). \qquad \text{(trivial)}$$

In the shortest path problem, we have

$$g_N(x_N) = 0.$$

Finite horizon deterministic problem

Backward computation:

$$J_N^*(x_N) = g_N(x_N). \qquad \text{(trivial)}$$

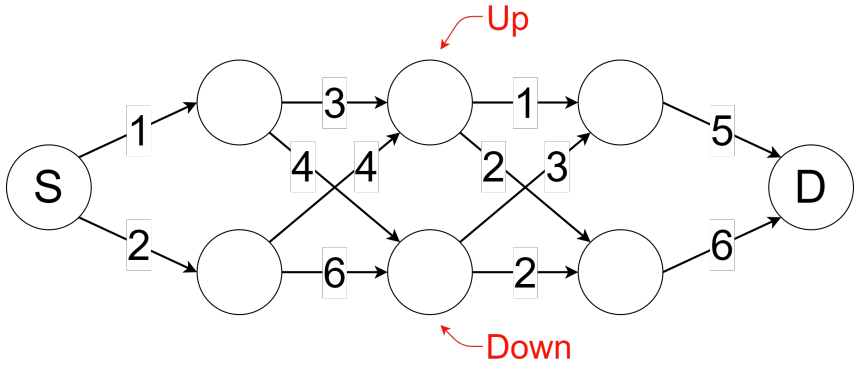
According to the DP principle,

$$J_{N-1}^*(x_{N-1}) = \min_{u_{N-1}} \left(g_{N-1}(x_{N-1}, u_{N-1}) + J_N^*(f(x_{N-1}, u_{N-1})) \right),$$

where we assume $f_k(\cdot) = f(\cdot)$ for all K. And in general,

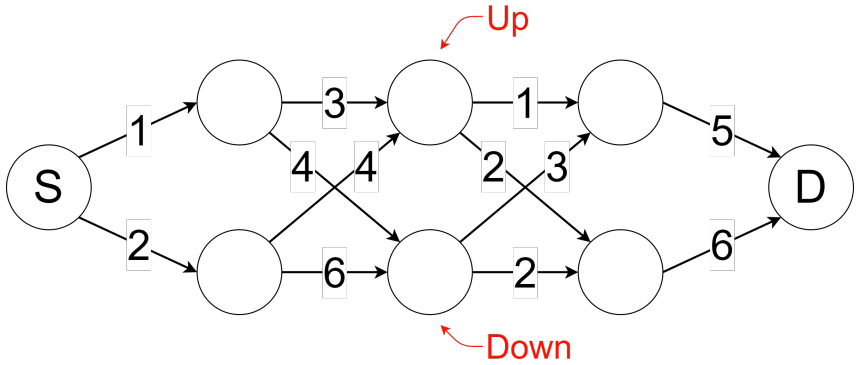
$$J_k^*(x_k) = \min_{u_k} \left(g_k(x_k, u_k) + J_{k+1}^*(f(x_k, u_k)) \right).$$

Finite horizon stochastic problem



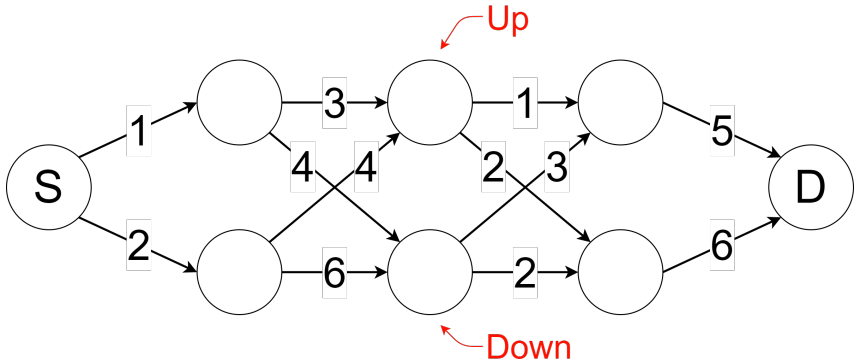
$$u_k = \text{up} \rightarrow \begin{cases} P(x_{k+1} = \text{top}) = 0.6 \\ P(x_{k+1} = \text{bottom}) = 0.4 \end{cases} \quad \text{and } u_k = \text{down} \rightarrow \begin{cases} P(x_{k+1} = \text{top}) = 0.4 \\ P(x_{k+1} = \text{bottom}) = 0.6 \end{cases}$$

Stochastic DP example



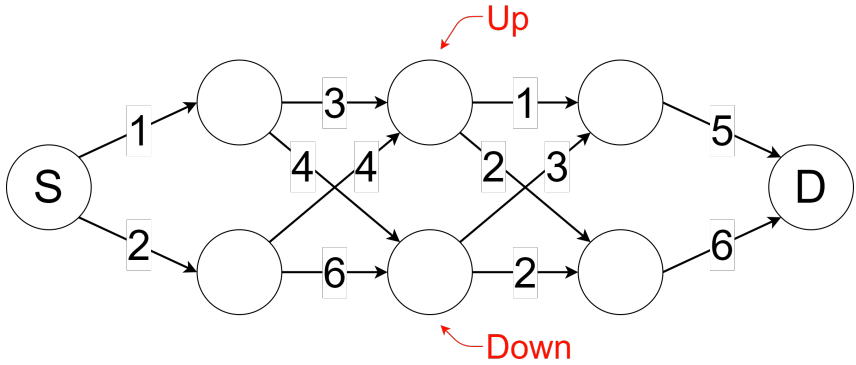
Optimal policy: A fixed sequence $(u_0, u_1, \ldots, u_{N-1})$?

Stochastic DP example



Optimal policy: A fixed sequence $(u_0, u_1, \ldots, u_{N-1})$?
NO.

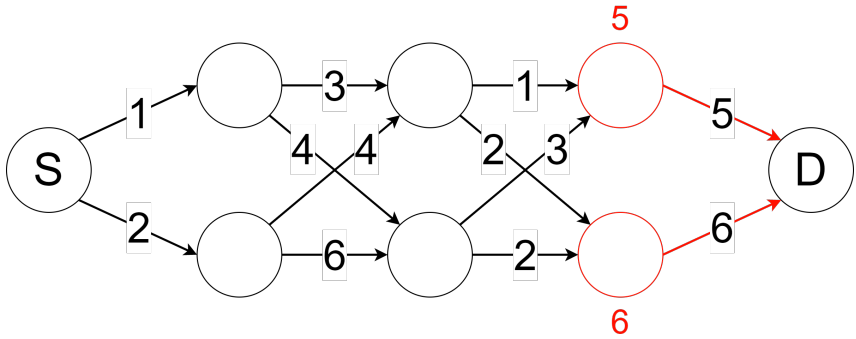
Stochastic DP example



Optimal policy: A fixed sequence $(u_0, u_1, \ldots, u_{N-1})$?

State dependent policy: $\mu_k^*(x_k)$, $\forall x_k$

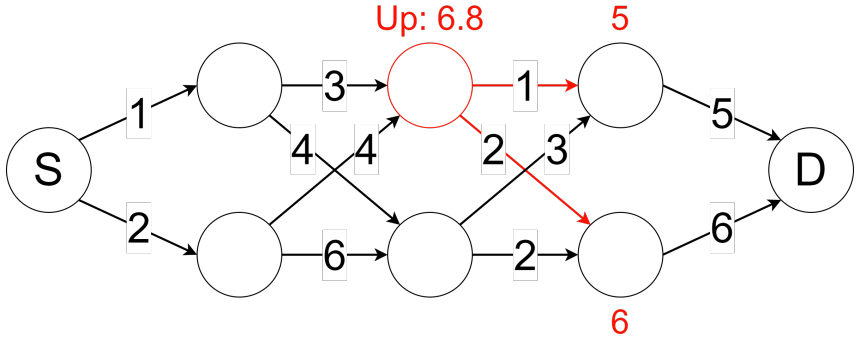
Stochastic DP example



The red nodes have only one possible path to reach node D.

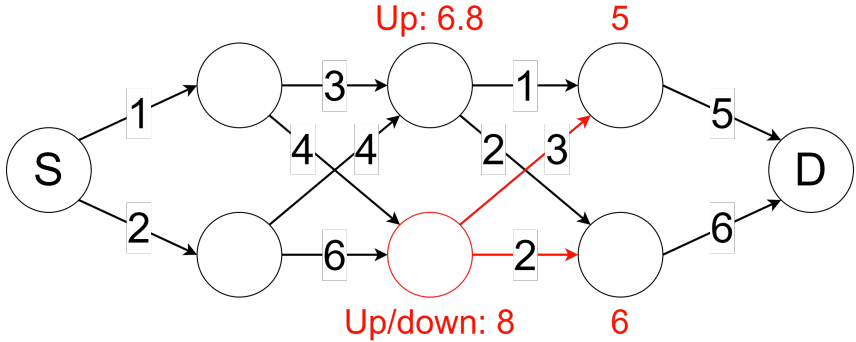
- Cost-to-go from the top node: 5
- Cost-to-go from the bottom node: 6

Stochastic DP example



- Cost-to-go with action Up: $0.6 \times (1+5) + 0.4 \times (2+6) = 3.6 + 3.2 = 6.8$
- Cost-to-go with action Down: $0.4 \times 6 + 0.6 \times 8 = 7.2$
- $\min\{\text{up}, \text{down}\} = 6.8 \implies u^*_{N-2}(\text{top}) = \textbf{up}$

Stochastic DP example



- Cost-to-go (Up): $0.6 \times (3 + 5) + 0.4 \times (2 + 6) = 8$
- Cost-to-go (Down): $0.4 \times 8 + 0.6 \times 8 = 8$
- $\min\{\text{up}, \text{down}\} = 8 \implies u^*_{N-2}(\text{bottom})$ can be either up or down

Stochastic DP

- Random transitions $P(x_{k+1}|x_k, u_k)$
- Cost function:

$$E\left[g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k)\right]$$

- Given initial state x_0, find a policy

$$\pi = \{\mu_0, \mu_1(x_1), \ldots, \mu_{N-1}(x_{N-1})\}$$

that minimizes

$$J(x_0) = \min_{\pi} E\left[g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k))\right]$$

Stochastic DP

The Bellman Equation:

$$\begin{aligned} J_k^*(x_k) &= \min_{u_k} E\left[g_k(x_k, u_k) + J_{k+1}^*(x_{k+1})\right] \\ &= \min_{u_k} \sum_g gP(g_k = g|x_k, u_k) + \sum_x J_{k+1}^*(x)P(x_{k+1} = x|x_k, u_k) \end{aligned}$$

Given state x_k and $J_{k+1}^*(x_{k+1})$,

$$u_k^* \in \arg\min_{u_k} E[g_k(x_k, u_k) + J_{k+1}^*(x_{k+1})]$$

Backward-forward algorithm: First compute J_k^* backward and then find u_k^* forward.

Stochastic Control versus Reinforcement Learning

- Transition kernel $P(x_{k+1}|x_k, u_k)$
- Cost function: $g_k(x_k, u_k)$
- Given initial state x_0, find a policy μ such that

$$J_0(x_0) = \min_{\mu} E\left[g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k))\right]$$

- Transition kernel $P(x_{k+1}|x_k, u_k)$
- Reward function: $r_k(x_k, u_k)$
- Given initial state x_0, find a policy μ such that

$$V_0(x_0) = \max_{\mu} E\left[r_N(x_N) + \sum_{k=0}^{N-1} r_k(x_k, \mu_k(x_k))\right]$$

Infinite Horizon Discounted RL

Time-homogeneous stochastic systems (environments)

- Transition kernel $P(x_k = j | x_{k-1} = i) = P(x' = j | x = i)$ does not depend on k
- Reward function $r_k(x_k = x, u_k = u) = r(x, u)$ does not depend on k
- Discount factor $0 < \gamma < 1$ and initial state x_0, find a policy μ such that

$$V_0(x_0) = \max_{\mu} E\left[\sum_{k=0}^{\infty} \gamma^k r(x_k, \mu_k(x_k))\right]$$

The Bellman Equation

$$V_k^*(x_k) = \max_{\mu_k} E\left[r(x_k, \mu_k(x_k)) + \gamma V_{k+1}^*(x_{k+1})\right]$$

$$\Longrightarrow V^*(x) = \max_{\mu} E\left[r(x, \mu(x)) + \gamma V^*(x')\right]$$

The Bellman Equation in RL

Episodic RL

$$V_k^*(x_k) = \max_{\mu_k} E\left[r_k(x_k, \mu_k(x_k)) + V_{k+1}^*(x_{k+1})\right]$$

Infinite Horizon Discounted RL

$$V^*(x) = \max_{\mu} E\left[r(x, \mu(x)) + \gamma V^*(x')\right]$$

The goal of reinforcement learning: Learn the optimal policy μ^* by solving the Bellman equation.

Value Iteration and Policy Iteration for Solving the Bellman Equation

Value iteration

- Recall the Bellman equation for infinite horizon discounted RL

$$V^*(x) = \max_a E\left[r(x,a) + \alpha V^*(x')\right].$$

Value iteration

- Recall the Bellman equation for infinite horizon discounted RL

$$V^*(x) = \max_a E\left[r(x,a) + \alpha V^*(x')\right].$$

- Suppose that both the state space $\mathcal{X}$ and action space $\mathcal{A}$ are finite, $P_{x,x'}$ and $\bar{r}(x,a) = E[r(x,a)]$ are known (model-based).

Value iteration

- Recall the Bellman equation for infinite horizon discounted RL

$$V^*(x) = \max_a E\left[r(x,a) + \alpha V^*(x')\right].$$

- Suppose that both the state space $\mathcal{X}$ and action space $\mathcal{A}$ are finite, $P_{x,x'}$ and $\bar{r}(x,a) = E[r(x,a)]$ are known (model-based).
- Starting from V_0, we can iteratively compute

$$V_{k+1} \leftarrow \max_a \left(\bar{r}(x,a) + \alpha \sum_{x'} P_{x,x'}(a) V_k(x')\right).$$

Value iteration

Starting from V_0, we can iteratively compute

$$V_{k+1} \leftarrow \max_a \left(\bar{r}(x,a) + \alpha \sum_{x'} P_{x,x'}(a) V_k(x') \right).$$

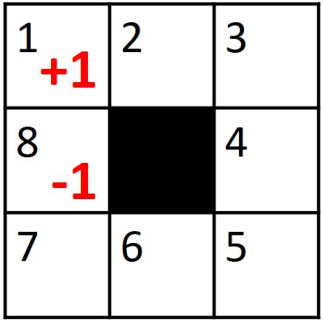
$$V_k \to V^*$$

After finding V^*, we can find the optimal policy by solving

$$\arg\max_a \left(\bar{r}(x,a) + \alpha \sum_{x'} P_{x,x'}(a) V^*(x') \right).$$

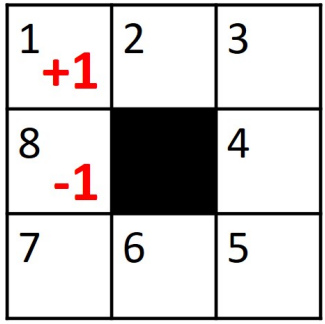
Value iteration: Grid world example

- Two actions: clockwise (c) or counter-clockwise (cc).



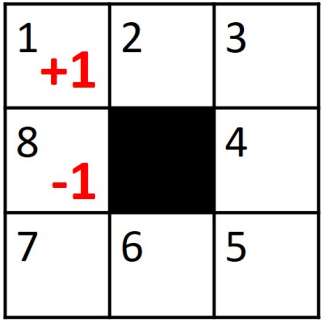
Value iteration: Grid world example

- Two actions: clockwise (c) or counter-clockwise (cc).
- The agent follows the action with probability 0.8 and moves to the opposite direction with probability 0.2.



Value iteration: Grid world example

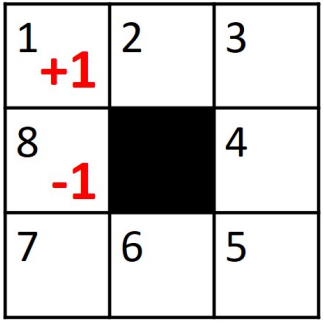
- Two terminal states with nonzero rewards: $+1$ in state 1, -1 in state 8. So for example, $r(2, cc) = 1$ with probability 0.8 and $r(2, cc) = 0$ with probability 0.2.



Value iteration: Grid world example

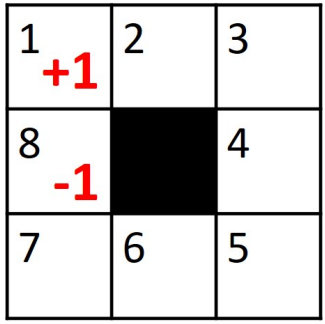
$V_k(1) = V_k(8) = 0$ for all k. Define

$$\mathbf{v}_k = \begin{pmatrix} V_k(2) \\ V_k(3) \\ V_k(4) \\ V_k(5) \\ V_k(6) \\ V_k(7) \end{pmatrix} \text{ and assume } V_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$



Value iteration: Grid world example

$$V_1(2) = \max_{a \in \{c, cc\}} E\left[r(2, a) + 0.9 V_0(x')\right]$$

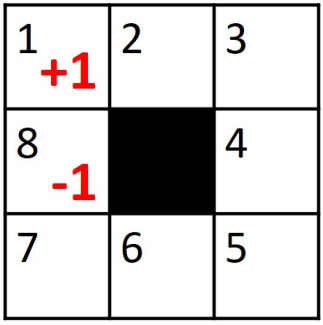


Value iteration: Grid world example

$$V_1(2) = \max_{a \in \{c, cc\}} E\left[r(2, a) + 0.9 V_0(x')\right]$$

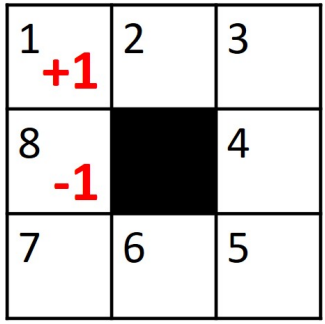
$$E[r(2, cc) + 0.9 V_0(x')] = 0.8 \times 1 + 0.2 \times 0 = 0.8$$

$$E[r(2, c) + 0.9 V_0(x')] = 0.2 \times 1 + 0.8 \times 0 = 0.2$$



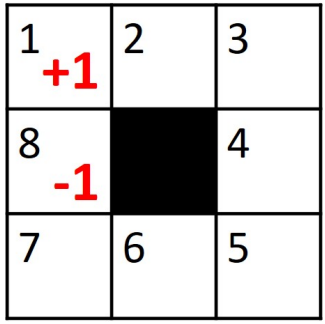
Value iteration: Grid world example

$$V_1(2) = \max_{a \in \{c, cc\}} E\left[r(2, a) + 0.9 V_0(x')\right]$$
$$= \max\{0.8, 0.2\} = 0.8.$$



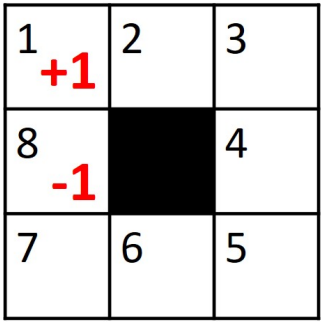
Value iteration: Grid world example

$$V_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \rightarrow V_1 = \begin{pmatrix} 0.8 \\ 0 \\ 0 \\ 0 \\ 0 \\ -0.2 \end{pmatrix} \rightarrow V_2 = \begin{pmatrix} 0.8 \\ 0.576 \\ 0 \\ 0 \\ -0.036 \\ -0.2 \end{pmatrix}$$



Value iteration: Grid world example

$$V^* = \begin{pmatrix} 0.9444 \\ 0.8023 \\ 0.6796 \\ 0.5661 \\ 0.4269 \\ 0.1074 \end{pmatrix}$$



Policy iteration

- Choose an initial policy μ, set $k = 0$
- Compute the value function of the policy V_{μ_k} by solving the Bellman equation.

$$V_{\mu_k}(i) = E[r(i,u)] + \alpha \sum_j P_{ij}(u) V_{\mu_k}(j)$$

- Compute a new policy μ_{k+1} as

$$\mu_{k+1}(i) \leftarrow \arg\max_u E[r(i,u)] + \alpha \sum_j P_{ij}(u) V_{\mu_k}(j)$$

- Repeat if $V_{\mu_{k+1}} \neq V_{\mu_k}$

Policy iteration

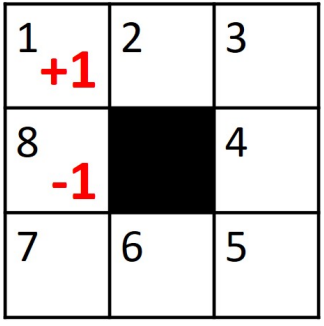
$V_{\mu_{k+1}} \geq V_{\mu_k}$, i.e. the value function improves at each step before the algorithm terminates.

Policy iteration: Grid world example

$\mu_0(i) = c$, i.e. the initial policy is to move clockwise in any state.

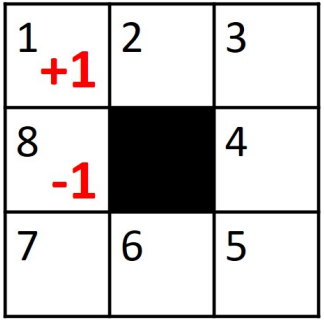
$$V_{\mu_0}(i) = E[r(i,c)] + 0.9\,(0.8\,V_{\mu_0}(i+1) + 0.2\,V_{\mu_0}(i-1))$$

$$V_{\mu_0} = \begin{pmatrix} -0.1074 \\ -0.4269 \\ -0.5661 \\ -0.6796 \\ -0.8023 \\ -0.9444 \end{pmatrix}$$



Policy iteration: Grid world example

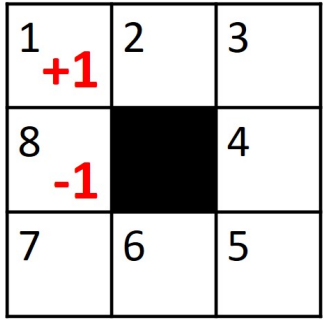
$\mu_1(i) = cc$, i.e. the improved policy is to go counter-clockwise (the optimal policy)



Policy iteration: Grid world example

$\mu_1(i) = cc$, i.e. the improved policy is to go counter-clockwise (the optimal policy)

$$\mu_1(4) = \arg\max_{cc,c}\{0.9 \times (0.8 \times -0.5661 + 0.2 \times -06796),$$
$$0.9 \times (0.2 \times -0.5661 + 0.8 \times -06796)\} = cc$$



Challenges of Applying These Algorithms

Challenges

Model-based algorithms: need to know $\bar{r}(i, u)$ and $P_{ij}(u)$

Challenges of Applying These Algorithms

Challenges

Model-based algorithms: need to know $\bar{r}(i, u)$ and $P_{ij}(u)$

Scalability: only work for finite state space and finite action space

Learning is needed when the system model is unknown.

Q-learning (Watkins '89)

Define Q-function (also called action value function):

$$Q(i, u) = \bar{r}(i, u) + \alpha \sum_j P_{ij}(u) V^*(j).$$

Q-learning (Watkins '89)

Define Q-function (also called action value function):

$$Q(i, u) = \bar{r}(i, u) + \alpha \sum_j P_{ij}(u) V^*(j).$$

Given Q, we can find the optimal policy by taking

$$\max_u Q(i, u)$$

(Note: does not require the model $P_{ij}(u)$)

Q-learning (Watkins '89)

Define Q-function (also called action value function):

$$Q(i, u) = \bar{r}(i, u) + \alpha \sum_j P_{ij}(u) V^*(j).$$

Given Q, we can find the optimal policy by taking

$$\max_u Q(i, u)$$

(Note: does not require the model $P_{ij}(u)$)

Q-learning: A learning algorithm to learn the Q-function.

Q-learning (Watkins '89)

Q-learning (off-policy)

Let $\tilde{Q}$ be the estimate of Q. Given an experience (current state $x_k = i$, current action $a_t = u$, received reward $r(i,u)$, and next state $x_{k+1} = j$), Q-learning updates $\tilde{Q}$ as follows:

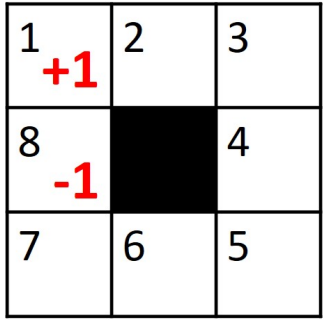
$$\begin{aligned}\tilde{Q}(i,u) \longleftarrow & (1-\beta_k)\tilde{Q}(i,u) + \beta_k(r(i,u) + \alpha \max_v \tilde{Q}(j,v)) \\ = & \tilde{Q}(i,u) + \beta_k \left(r(i,u) + \alpha \max_v \tilde{Q}(j,v) - \tilde{Q}(i,u)\right)\end{aligned}$$

$$Q(i,u) = \bar{r}(i,u) + \alpha \sum_j P_{ij}(u) V^*(j) = \bar{r}(i,u) + \alpha \sum_j P_{ij}(u) \max_v Q(j,v).$$

Q-Learning: Grid world example

Assume the current estimate is

(2,c) 0.4	(3,c) 0.38	(4,c) 0.14	(5,c) 0.2	(6,c) -0.8	(7,c) -0.9
(2,cc) 0.85	(3,cc) 0.78	(4,cc) 0.34	(5,cc) 0.3	(6,cc) -0.5	(7,cc) -0.34



Q-Learning: Grid world example

Assume the current estimate is

(2,c) 0.4	(3,c) 0.38	(4,c) 0.14	(5,c) 0.2	(6,c) -0.8	(7,c) -0.9
(2,cc) 0.85	(3,cc) 0.78	(4,cc) 0.34	(5,cc) 0.3	(6,cc) -0.5	(7,cc) -0.34

Given experience $(x_k = 5, u_k = c, r_k = 0, x_{k+1} = 6)$, Q-learning updates $\tilde{Q}(5, c)$

$$\tilde{Q}(5, c) \longleftarrow 0.2 + \beta_k \left(0 + \alpha \max\{-0.8, -0.5\} - 0.2\right)$$

$$\tilde{Q}(i, u) \longleftarrow \tilde{Q}(i, u) + \beta_k \left(r(i, u) + \alpha \max_v \tilde{Q}(j, v) - \tilde{Q}(i, u)\right)$$

State–action–reward–state–action (SARSA) (Rummery& Niranjan'94)

SARSA (on-policy)

- At step k, with probability $1 - \epsilon_k$, choose action u_k such that

$$u_k \in \arg\max_v \tilde{Q}(x_k, v)$$

  and with probability ϵ_k, choose an action u_k uniformly at random. Observe x_{k+1} and $r(x_k, u_k)$.
- With data $(x_{k-1}, u_{k-1}, x_k, u_k)$, update Q such that

$$\tilde{Q}(x_{k-1}, u_{k-1}) \longleftarrow (1 - \beta_k)\tilde{Q}(x_{k-1}, u_{k-1}) + \beta_k(r(x_{k-1}, u_{k-1}) + \alpha\tilde{Q}(x_k, u_k))$$

- Choose $\{\epsilon_k\}$ such that $\epsilon_k \to 0$ as $k \to \infty$

SARSA: Grid world example

Assume at time k, the esimate is

(2,c)	(3,c)	(4,c)	(5,c)	(6,c)	(7,c)
0.4	0.38	0.14	0.2	-0.8	-0.9
(2,cc)	(3,cc)	(4,cc)	(5,cc)	(6,cc)	(7,cc)
0.85	0.78	0.34	0.3	-0.5	-0.34

and assume $x_{k-1} = 4$, $u_{k-1} = cc$, $r_{k-1} = 0$, and $x_k = 5$.

SARSA: Grid world example

Assume at time k, the esimate is

(2,c) 0.4	(3,c) 0.38	(4,c) 0.14	(5,c) 0.2	(6,c) -0.8	(7,c) -0.9
(2,cc) 0.85	(3,cc) 0.78	(4,cc) 0.34	(5,cc) 0.3	(6,cc) -0.5	(7,cc) -0.34

and assume $x_{k-1} = 4$, $u_{k-1} = cc$, $r_{k-1} = 0$, and $x_k = 5$.
SARSA chooses action:

$$u_k = \arg\max_{c,cc} \{\tilde{Q}(5, c) = 0.2, \tilde{Q}(5, cc) = 0.3\} = cc.$$

SARSA: Grid world example

Assume at time k, the esimate is

(2,c) 0.4	(3,c) 0.38	(4,c) 0.14	(5,c) 0.2	(6,c) -0.8	(7,c) -0.9
(2,cc) 0.85	(3,cc) 0.78	(4,cc) 0.34	(5,cc) 0.3	(6,cc) -0.5	(7,cc) -0.34

and assume $x_{k-1} = 4$, $u_{k-1} = cc$, $r_{k-1} = 0$, and $x_k = 5$.
SARSA updates $\tilde{Q}(4, cc)$:

$$\tilde{Q}(4, cc) \longleftarrow 0.34 + \beta_k \left(0 + \alpha \times 0.3 - 0.34\right).$$

$$\tilde{Q}(x_{k-1}, u_{k-1}) \longleftarrow (1 - \beta_k)\tilde{Q}(x_{k-1}, u_{k-1}) + \beta_k(r(x_{k-1}, u_{k-1}) + \alpha\tilde{Q}(x_k, u_k))$$

Off-policy vs. on-policy reinforcement learning

Target policy: The policy to be learned

Behavior policy: The policy used to generate samples

- Q-learning: target policy - optimal policy
  behavior policy - any policy under which each action is taken infinitely often
- SARSA: target policy - ϵ-greedy
  behavior policy - ϵ-greedy

Reinforcement Learning with Function Approximations

Challenges in solving the Bellman equation:

- Model-based algorithms: need to know $\bar{r}(i, u)$ and $P_{ij}(u)$
  $\rightarrow$ model free, data-driven methods. Q-Learning, SARSA, etc

Reinforcement Learning with Function Approximations

Challenges in solving the Bellman equation:

- Model-based algorithms: need to know $\bar{r}(i, u)$ and $P_{ij}(u)$
  $\rightarrow$ model free, data-driven methods. Q-Learning, SARSA, etc
- Scalability or Curse-of-dimensionality: large state and action spaces (the 9×9 Go game has 10^{35} states).
  $\rightarrow$ function approximation. Linear Function Approximation or Deep Neural Networks

Linear Function Approximation (Feature-Based)

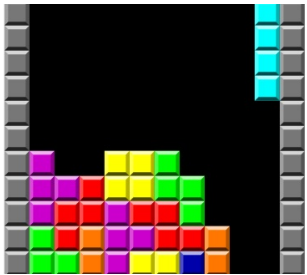
$$Q(i,u) \approx \sum_{m=1}^{M} \theta_m \phi_m(i,u) = \theta^T \phi(i,u)$$

- $\phi(i,u)$: M-dimensional feature vector for (i,u) and is assumed to be given
- θ: M-dimensional weight vector; common for all (state, action) pairs.

Linear Function Approximation (Feature-Based)

$$Q(i,u) \approx \sum_{m=1}^{M} \theta_m \phi_m(i,u) = \theta^T \phi(i,u)$$

- $\phi(i,u)$: M-dimensional feature vector for (i,u) and is assumed to be given
- θ: M-dimensional weight vector; common for all (state, action) pairs.



Features: the height of the highest column, differences in height between neighboring columns, holes, and wells

Picture is from `https://en.wikipedia.org/wiki/Tetris`

Linear Function Approximation (Feature-Based)

$$Q(i,u) \approx \sum_{m=1}^{M} \theta_m \phi_m(i,u) = \theta^T \phi(i,u)$$

- $\phi(i,u)$: M-dimensional feature vector for (i,u) and is assumed to be given
- θ: M-dimensional weight vector; common for all (state, action) pairs.

Learning $Q(i,u)$ ($|\mathcal{S}| \times |\mathcal{A}|$) pairs) is equivalent to learning θ.

- If $M << |\mathcal{S}| \times |\mathcal{A}|$, the complexity reduces significantly.
- Learning θ also allows us to generalize when facing new state-action pair.

Q-Learning with Linear Function Approximation

Temporal Difference

$$\delta_k = r(x_k, u_k) + \alpha \max_v \tilde{Q}(x_{k+1}, v) - \tilde{Q}(x_k, u_k)$$

Q-Learning with Linear Function Approximation

Temporal Difference

$$\delta_k = r(x_k, u_k) + \alpha \max_v \tilde{Q}(x_{k+1}, v) - \tilde{Q}(x_k, u_k)$$

Temporal Difference with Linear Function Approximation

$$\delta_k(\tilde{\theta}) = r(x_k, u_k) + \alpha \max_v \tilde{\theta}^T \phi(x_{k+1}, v) - \tilde{\theta}^T \phi(x_k, u_k)$$

Q-Learning with Linear Function Approximation

Temporal Difference

$$\delta_k = r(x_k, u_k) + \alpha \max_v \tilde{Q}(x_{k+1}, v) - \tilde{Q}(x_k, u_k)$$

Q-Learning

$$\tilde{Q}(x_k, u_k) \leftarrow \tilde{Q}(x_k, u_k) + \beta_k \left(r(x_k, u_k) + \alpha \max_v \tilde{Q}(x_{k+1}, v) - \tilde{Q}(x_k, u_k) \right)$$

$$\tilde{Q}(x_k, u_k) \leftarrow \tilde{Q}(x_k, u_k) + \beta_k \delta_k \approx \tilde{Q}(x_k, u_k) - \frac{1}{2} \beta_k \frac{\partial \delta_k^2}{\partial \tilde{Q}(x_k, u_k)}$$

Q-Learning with Linear Function Approximation

Temporal Difference with Linear Function Approximation

$$\delta_k(\tilde{\theta}) = r(x_k, u_k) + \alpha \max_v \tilde{\theta}^T \phi(x_{k+1}, v) - \tilde{\theta}^T \phi(x_k, u_k)$$

Q-Learning with Linear Function Approximation

Temporal Difference with Linear Function Approximation

$$\delta_k(\tilde{\theta}) = r(x_k, u_k) + \alpha \max_v \tilde{\theta}^T \phi(x_{k+1}, v) - \tilde{\theta}^T \phi(x_k, u_k)$$

Q-Learning with Linear Function Approximation

$$\tilde{\theta} \leftarrow \tilde{\theta} - \frac{1}{2}\beta_k \frac{\partial \delta_k^2(\tilde{\theta})}{\partial \tilde{\theta}}$$

Q-Learning with Linear Function Approximation

Temporal Difference with Linear Function Approximation

$$\delta_k(\tilde{\theta}) = r(x_k, u_k) + \alpha \max_v \tilde{\theta}^T \phi(x_{k+1}, v) - \tilde{\theta}^T \phi(x_k, u_k)$$

Q-Learning with Linear Function Approximation

$$\tilde{\theta} \leftarrow \tilde{\theta} - \frac{1}{2}\beta_k \frac{\partial \delta_k^2(\tilde{\theta})}{\partial \tilde{\theta}}$$

Q-Learning with Linear Function Approximation

$$\tilde{\theta} \leftarrow \tilde{\theta} + \beta_k \delta_k(\tilde{\theta}) \phi(x_k, u_k)$$

Deep Reinforcement Learning Algorithms

Deep Q-Learning

Approximate $Q(x, u)$ using a neural network: $\hat{Q}(x, u; \theta)$, where θ is the weights of a neural network.

Deep Q-Learning

Approximate $Q(x, u)$ using a neural network: $\hat{Q}(x, u; \theta)$, where θ is the weights of a neural network.

Learning objective

$$\min_{\theta} E\left[\frac{1}{2}\left(r(x_k, u_k) + \alpha \max_{v} \hat{Q}(x_{k+1}, v; \theta) - \hat{Q}(x_k, u_k; \theta)\right)^2\right]$$

Deep Q-Learning

Approximate $Q(x, u)$ using a neural network: $\hat{Q}(x, u; \theta)$, where θ is the weights of a neural network.

Learning objective

$$\min_{\theta} E\left[\frac{1}{2}\left(r(x_k, u_k) + \alpha \max_{v} \hat{Q}(x_{k+1}, v; \theta) - \hat{Q}(x_k, u_k; \theta)\right)^2\right]$$

Q-learning with neural networks

$$\theta \leftarrow \theta + \beta_k \delta_k \nabla_\theta \hat{Q}(x_k, u_k; \theta),$$

where

$$\delta_k = r(x_k, u_k) + \alpha \max_{v} \hat{Q}(x_{k+1}, v; \theta) - \hat{Q}(x_k, u_k; \theta)$$

Deep Q-Learning

Issues of deep reinforcement learning

No theoretical guarantees on optimality and convergence
More unstable during training

Replay Buffer and Experience Replay

Pretend we have a supervised learning problem

$$\frac{1}{2} \min_{\theta} \sum_{k} \Big(y_k - \hat{Q}(x_k, u_k; \theta) \Big)^2,$$

where $y_k = r(x_k, u_k) + \alpha \max_v \hat{Q}(x_{k+1}, v; \theta)$ (we ignore the fact that y_k is a function of θ like in linear function approximation.)

Replay Buffer and Experience Replay

Pretend we have a supervised learning problem

$$\frac{1}{2}\min_{\theta}\sum_{k}\Big(y_k-\hat{Q}(x_k,u_k;\theta)\Big)^2,$$

where $y_k = r(x_k,u_k)+\alpha\max_v \hat{Q}(x_{k+1},v;\theta)$ (we ignore the fact that y_k is a function of θ like in linear function approximation.)

Stochastic Gradient Descent (SGD)

$$\theta \leftarrow \theta+\beta_k(y_k-\hat{Q}(x_k,u_k;\theta))\nabla_\theta\hat{Q}(x_k,u_k;\theta)$$

with *independently and identically distributed* data samples $(y_k,(x_k,u_k))$.

Replay Buffer and Experience Replay

Stochastic Gradient Descent (SGD)

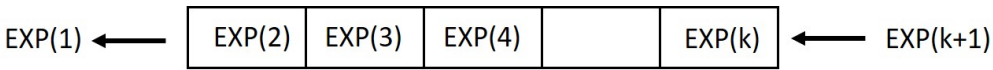
$$\theta \leftarrow \theta + \beta_k(y_k - \hat{Q}(x_k, u_k; \theta))\nabla_\theta \hat{Q}(x_k, u_k; \theta)$$

with *independently and identically distributed* data samples $(y_k, (x_k, u_k))$.

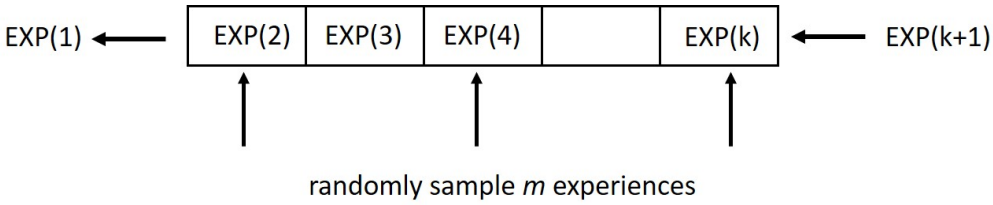
$$(y_1, (x_1, u_1)) \to (y_2, (x_2, u_2)) \to (y_3, (x_3, u_3)) \to \cdots$$

are highly correlated samples in reinforcement learning applications.

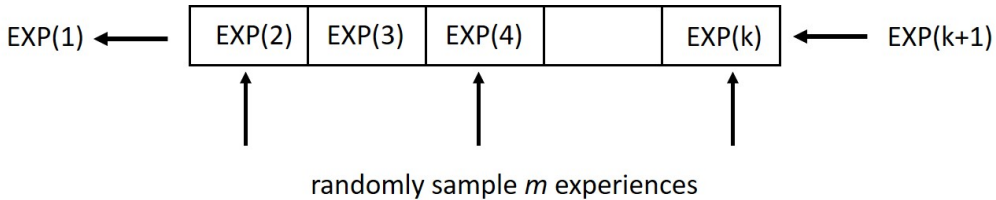
Replay Buffer



Replay Buffer and Experience Replay



Replay Buffer and Experience Replay



Minibatch SGD

$$\theta \leftarrow \theta + \beta_k \sum_{b=1}^{m} (y_{k,b} - \hat{Q}(x_{k,b}, u_{k,b}; \theta)) \nabla_\theta \hat{Q}(x_{k,b}, u_{k,b}; \theta).$$

Target Network

From the supervised learning perspective, the target value $y_{k,m} = r(x_{k,m}, u_{k,m}) + \alpha \max_v \hat{Q}(x_{k+1,m}, v; \theta)$ should be independent of θ.

Target Network

From the supervised learning perspective, the target value $y_{k,m} = r(x_{k,m}, u_{k,m}) + \alpha \max_v \hat{Q}(x_{k+1,m}, v; \theta)$ should be independent of θ.

Target Network Q_{target}

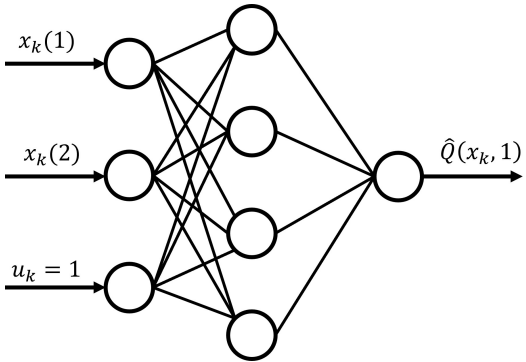
copy θ to θ_{target} once every n iterations and use Q_{target} to calculate $y_{k,m}$.

Deep Q-Network (DQN)

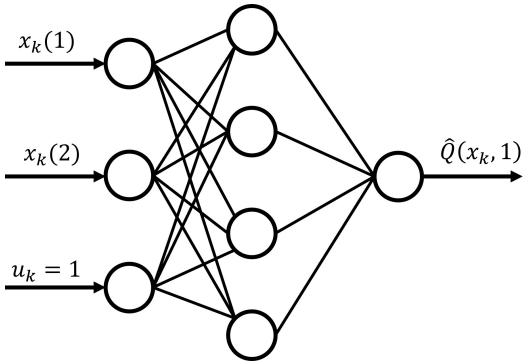
How to calculate $\sum_{b=1}^{m}(y_{k,b} - \hat{Q}(x_{k,b}, u_{k,b}; \theta))\nabla_\theta \hat{Q}(x_{k,b}, u_{k,b}; \theta)$ and update θ?

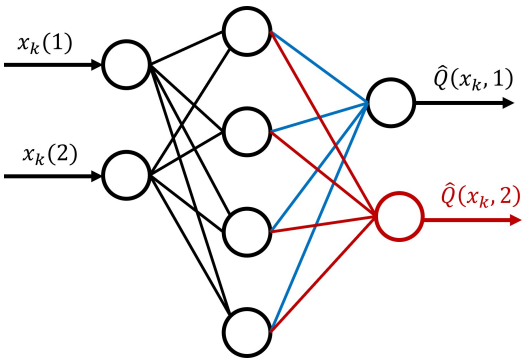
Deep Q-Learning in PyTorch

Deep Q-Network Architecture



Deep Q-Network Architecture

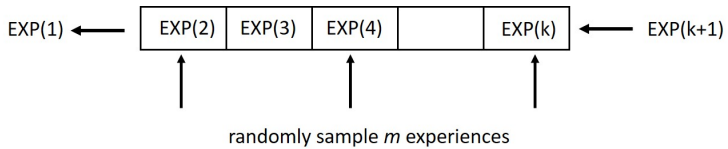




Couple the estimation of Q-values ($Q(x, u)$) of different actions u for the same state x.

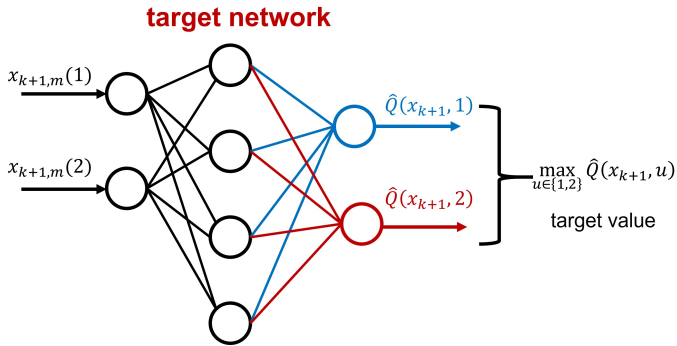
Deep-Q-Network (DQN) (Minh et al 2015) in PyTorch

- Sample a mini-batch $\{(x_{k,b}, u_{k,b}, r_{k,b}, x_{k+1,b})\}_{b=1,\cdots,m}$.
  random.sample(self.replay_buffer, batch_size)



Deep-Q-Network (DQN) (Minh et al 2015) in PyTorch

- Generate target-values $y_{k,m}$ using the target network
  next_state_values = target_net(next_states).max(1)[0]
  target_values= rewards+ gamma × next_state_values.detach()

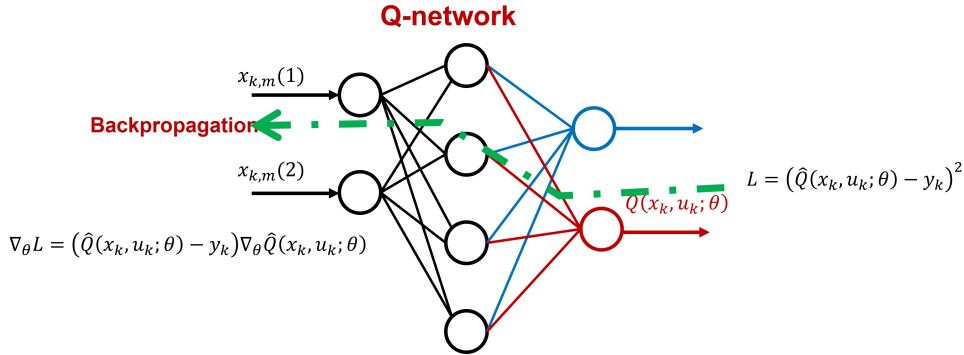


Deep-Q-Network (DQN) (Minh et al 2015) in PyTorch

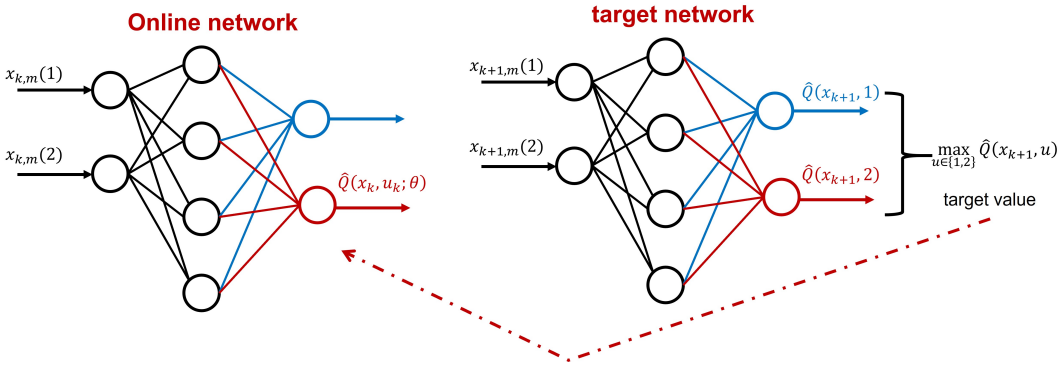
- Define a loss function $L(\theta) = \sum_{b=1}^{m} \left(\hat{Q}(x_{k,b}, u_{k,b}; \theta) - y_{k,b} \right)^2$.
  loss = nn.MSELoss()(state_action_values, target_values)

Deep-Q-Network (DQN) (Minh et al 2015) in PyTorch

- Update $\theta \leftarrow \theta - \beta_k \nabla_\theta L(\theta)$
  optimizer.zerograd() loss.backward() optimizer.step()

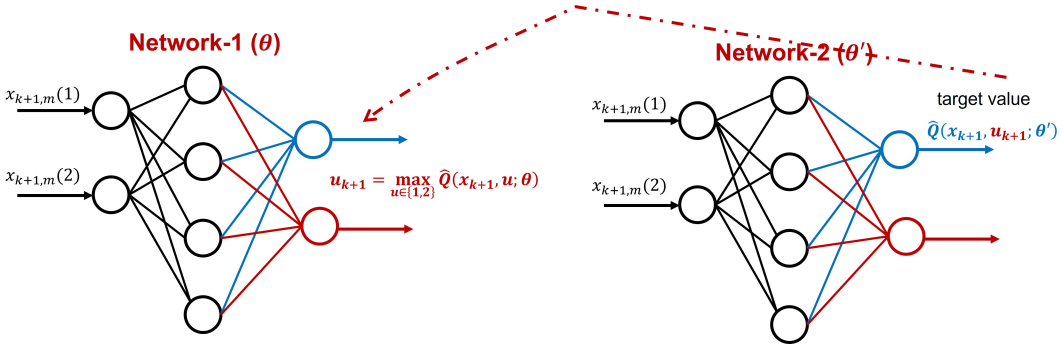


Double DQN (van Hasselt, Guez, Silver 2015)



Target network is used to (i) select the action u (ii) evaluate $Q(x_{k+1}, u)$. Selecting the optimal action and evaluating the Q-function together lead to an overestimation.

Double DQN (van Hasselt, Guez, Silver 2015)



Double DQN

Separate action selection and evaluation. Both Q-networks are trained at each iteration by using the other one as the target-network.

Clipped Double-Q Learning (Fujimoto, van Hoof, David Meger 2018)

Maintain two Q-networks, denoted by their parameters θ and θ',

$$\theta \leftarrow \theta + \beta_k \sum_b (y_{k,b} - Q(x_{k,b}, u_{k,b}; \theta)) \nabla_\theta Q(x_{k,b}, u_{k,b}; \theta)$$

$$\theta' \leftarrow \theta' + \beta_k \sum_b (y_{k,b} - Q(x_{k,b}, u_{k,b}; \theta')) \nabla_{\theta'} Q(x_{k,b}, u_{k,b}; \theta')$$

where

$$y_{k,b} = r_{k,b} + \alpha \min \left\{ \max_u Q(x_{k+1,b}, u; \theta), \max_u Q(x_{k+1,b}, u; \theta') \right\}$$

Policy Gradient and Actor-Critic Algorithms

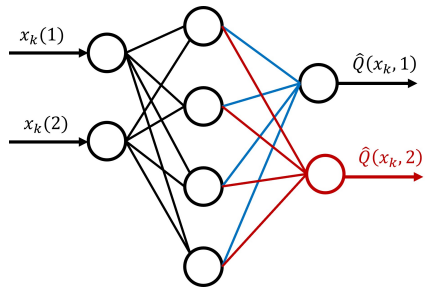
Q-learning follows value iteration

$$\tilde{Q}(i,u) \leftarrow \tilde{Q}(i,u) + \beta_k \left(r(i,u) + \alpha \max_v \tilde{Q}(j,v) - \tilde{Q}(i,u) \right)$$

Q-learning follows value iteration

$$\tilde{Q}(i,u) \leftarrow \tilde{Q}(i,u) + \beta_k \left(r(i,u) + \alpha \max_v \tilde{Q}(j,v) - \tilde{Q}(i,u) \right)$$

Works well when the action space is finite.



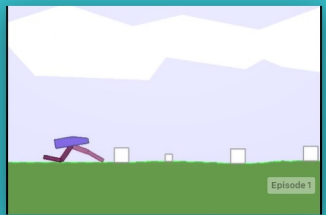
Policy Iteration

How about applications with infinite (countable or uncountable) action spaces?

BipedalWalkerHardcore-v2

Hardcore version with ladders, stumps, pitfalls. Time limit is increased due to obstacles. Reward is given for moving forward, total 300+ points up to the far end. If the robot falls, it gets -100. Applying motor torque costs a small amount of points, more optimal agent will get better score. State consists of hull angle speed, angular velocity, horizontal speed, vertical speed, position of joints and joints angular speed, legs contact with ground, and 10 lidar rangefinder measurements. There's no coordinates in the state vector.

VIEW SOURCE ON GITHUB



RandomAgent on BipedalWalkerHardcore-v2

Policy Iteration

How about applications with infinite (countable or uncountable) action spaces?

Policy Iteration

Given policy μ_k,

- policy evaluation to compute V_{μ_k}

$$V_{\mu_k}(i) = E[r(i,u)] + \alpha \sum_j P_{ij}(u) V_{\mu_k}(j)$$

- policy improvement to obtain a new policy μ_{k+1}

Deep Actor-Critic Algorithms

Actor-critic: generalized policy-iteration

- Critic: estimate the value of the current policy (Q-function in general, because the actor needs the action values)
  TD(λ), double-Q, clipped double-Q, etc.
- Actor:
  - ϵ-greedy based on the current Q-function
  - Policy-gradient

Actor-Critic

Parameterized Policy

$\pi_\theta(u|x)$: probability of choosing action u in state x. The policy (probability) is parameterized by θ.

Actor-Critic

Parameterized Policy

$\pi_\theta(u|x)$: probability of choosing action u in state x. The policy (probability) is parameterized by θ.

Given Q-function (action value function),

$$V_{\theta^*} = \max_\theta E\left[\sum_u Q(x,u)\pi_\theta(u|x)\right].$$

Examples

Finite action space: Gibbs policy

$$\pi_\theta(a|x) = \frac{\exp\left(\theta^T \phi(x,a)\right)}{\sum_{u \in A} \exp\left(\theta^T \phi(x,u)\right)}$$

where $\phi(x,u)$ is the feature vector.

Examples

Finite action space: Gibbs policy

$$\pi_\theta(a|x) = \frac{\exp\left(\theta^T \phi(x,a)\right)}{\sum_{u \in A} \exp\left(\theta^T \phi(x,u)\right)}$$

where $\phi(x,u)$ is the feature vector.

Continuous action space: Gaussian policy

$$\pi_\theta(a|x) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma_\theta(x))}} \exp\left\{-(a-\mu_\theta(x))^T \Sigma_\theta^{-1}(x)(a-\mu_\theta(x))\right\}$$

where $\mu_\theta(x) = \phi^T(x)\theta$ and $\Sigma_\theta(x) = \sigma^2 I$.

Policy Gradient

$$\theta \leftarrow \theta + \beta \nabla_\theta V_\theta$$

Policy Gradient Theorem

$$\nabla_\theta V_\theta = E_{x \sim \rho_\theta, a \sim \pi_\theta(a|x)} \left[Q^{\pi_\theta}(x, a) \nabla_\theta \log \pi_\theta(a|x) \right],$$

where $\rho_0(x)$ is the initial distribution of the states and

$$\rho_\theta(x) = \sum_{k=0}^{\infty} \alpha^k \Pr\left(x_k = x\right),$$

called (improper) discounted state distribution.

Actor-critic

$\nabla_\theta \log \pi_\theta(a|x)$: score function

- Gibbs policy: $\nabla_\theta \log \pi_\theta(a|x) = \phi(x, a) - E_{\pi_\theta}[\phi(x, \cdot)]$
- Gaussian policy: $\nabla_\theta \log \pi_\theta(a|x) = \frac{(a-\mu(x))\phi(x)}{\sigma^2}$ where $\mu(x) = \phi^T(x)\theta$

REINFORCE (Williams (1988, 1992))

Generate an episode $x_0, a_0, x_1, a_1, \cdots, x_{T-1}, a_{T-1}, x_T$ by following policy π_θ. For each step t of the episode, update θ as follows:

$$\theta \leftarrow \theta + \beta \alpha^t \nabla_\theta \log \pi_\theta(a_t|x_t) \sum_{\tau=t}^{T} \alpha^{\tau-t} r_t.$$

Continue to the next episode with the updated θ.

Monte-Carlo Policy

REINFORCE uses the Monte-Carlo method

$$Q_\theta(x_t, a_t) \approx \sum_{\tau=t}^{T} \alpha^{\tau-t} r_t.$$

Variance Reduction and Deep Policy-Gradient Algorithms

Variance Reduction (Control Variates Method)

We are interested in computing

$$E[f(x)] \approx \underbrace{\frac{1}{N}\sum_{i=1}^{N} f(x_i)}_{F} \quad x_i \sim P(x)$$

But F (finite-sample average) may have a high variance.

Variance Reduction

Replace F with F' such that

$$E[F] = E[F'],\ Var(F') \leq Var(F)$$

Variance Reduction

Consider function $\phi(x)$ such that $E[\phi(x)] = 0$,

$$E[F(x) - \phi(x)] = E[F(x)]$$

$$Var(F(x) - \phi(x)) = Var(F(x)) - 2Cov(F(x), \phi(x)) + Var(\phi(x))$$

The variance can be reduced when $\phi(x)$ is strongly correlated with $F(x)$.

Estimate $\nabla_\theta V(i)$

$$\nabla_\theta V(i) = \sum_{k=0}^{T} \alpha^k \nabla \log \pi_\theta(u_k|x_k) \times (\underbrace{r(x_k, u_k) + \alpha V_\theta(x_{k+1}) - V_\theta(x_k)}_{\text{TD error}}).$$

Actor-Critic

- Advantage Actor-Critic:

$$A(s, a) = Q_\theta(s, a) - V_{\tilde{\theta}}(s)$$

- TD Actor-Critic:

$$A(s, a) = r(s, a) + \alpha V_\theta(s') - V_\theta(s)$$

TD error estimates the advantage function.

Actor-Critic with Neural Networks

Critic

double-Q, target-Q, and clipped-Q

Actor-Critic with Neural Networks

Critic

double-Q, target-Q, and clipped-Q

Actor

Sample a mini-batch sampled from replay buffer. Train the policy network with a weighted cross-entropy loss:

$$L = -\sum_{(s,a)} A(s,a) \log \pi_\theta(a|s)$$

where

$$A(s,a) = r(s,a) + \alpha V_\theta(s') - V_\theta(s).$$

Deep deterministic policy gradient (DDPG) (Lillicrap et al. 2016)

Critic

Q-network $Q_w(s, a)$: trained with mini-batch and temporal-difference.

Deep deterministic policy gradient (DDPG) (Lillicrap et al. 2016)

Critic

Q-network $Q_w(s, a)$: trained with mini-batch and temporal-difference.

Actor

Deterministic policy-network $\mu_\theta(x)$: trained with loss function

$$L(\theta) = - \sum_{s_b \in \text{minibatch}} Q_w(s_b, \mu_\theta(s_b)).$$

Deep deterministic policy gradient (DDPG) (Lillicrap et al. 2016)

Critic

Q-network $Q_w(s, a)$: trained with mini-batch and temporal-difference.

Actor

Deterministic policy-network $\mu_\theta(x)$: trained with loss function

$$L(\theta) = - \sum_{s_b \in \text{minibatch}} Q_w(s_b, \mu_\theta(s_b)).$$

Twin Delayed DDPG (TD3) (Fujimoto, van Hoof, Meger, 2018): Clipped double-Q + deterministic PG.

Exploration vs. Exploitation

Exploration vs. Exploitation

Exploration: Try actions to find the best one.

Exploration vs. Exploitation

Exploration: Try actions to find the best one.

Exploitation: Take the action “believed” to be the best.

Exploration vs. Exploitation

Exploration: Try actions to find the best one.

Exploitation: Take the action "believed" to be the best.

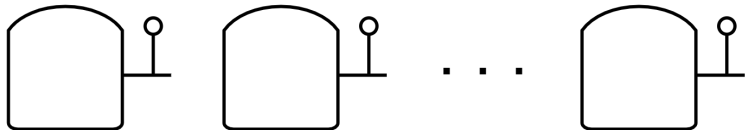
A straightforward strategy: ϵ-greedy.

Question

What is the optimal exploration and exploitation tradeoff?

Multi-armed bandit

Stochastic bandits with k possible arms



- Reward: $X_i(t)$ when arm i is played at time t. $X_i(t)$ is i.i.d random variable (for given i).
- Goal: $\max \sum_{t=1}^{T} E[X_{i_t}(t)]$
- i_t: arm played at time t.

ϵ-greedy

- With probability $(1 - \epsilon)$,

$$i^* \in \arg\max_i \mu_i(t)$$

  where

$$\mu_i(t) = \frac{\text{total reward received from playing arm } i}{\text{number of times arm } i \text{ has been played}}.$$

- With probability ϵ, randomly pick an arm.

Upper Confidence Bound (UCB) Algorithm

- Play each arm once at the beginning
- At time $t > K$, choose arm i^* such that

$$i^* \in \arg\max_i \mu_i(t) + \sqrt{\frac{\alpha \log(t)}{N_i(t)}}$$

  - $N_i(t)$: number of times arm i played by time t
  - $\mu_i(t) = \frac{\sum_{s=1}^{t} \mathbb{I}_{i(s)=i} X_{i(s)}(s)}{N_i(t)}$
  - $\sqrt{\frac{\alpha \log(t)}{N_i(t)}}$: confidence internal about estimating μ_i with current observations
    large $N_i(t)$: more confident about arm i
    small $N_i(t)$: less confident

Upper Confidence Bound (UCB) Algorithm

- Explore arms with more uncertain. Optimism in Face of Uncertainty.
- UCB balances exploration and exploitation
- As $t \to \infty$, the probability of selecting the best arm goes to one because

$$\sqrt{\frac{\alpha \log(t)}{N_i(t)}} \to 0, \text{ as } t \to \infty$$

UCB Algorithm

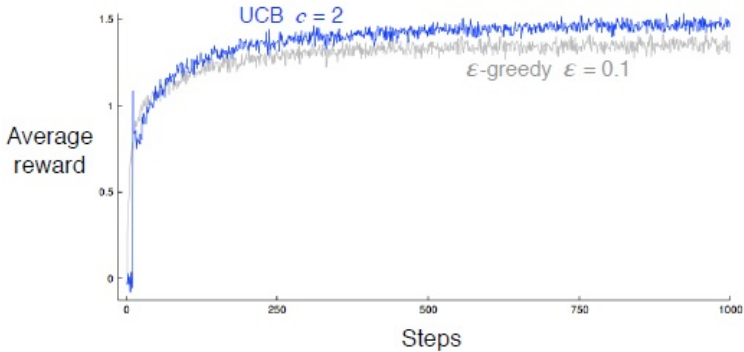


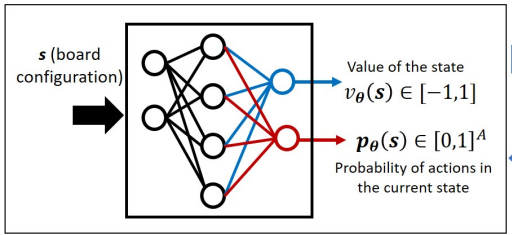
Figure: UCB versus ϵ-greedy (Figure 2.3 in Sutton and Barto)

AlphaZero[1] [2]

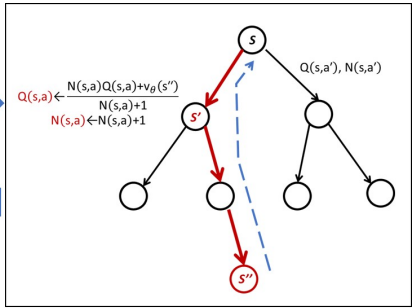
[1] Paper: D. Silver et al. "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm", Science, 2018.

[2] Tutorial: `https://web.stanford.edu/~surag/posts/alphazero.html`

Overview of AlphaZero



Learn the improved the policy with samples from self-play (similar to Critic)



Improve the policy using the policy-network and MCTS (similar to Actor)

Policy Network

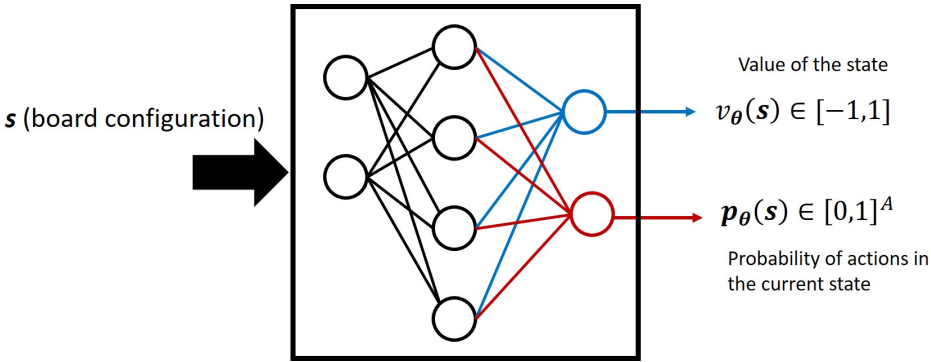


Figure: The Policy Network in AlphaZero

Training the Policy Network (Similar to Critic)

Training Samples

K plays of the game and $(s_{h,k}, \pi_{h,k}, z_k)$ is a sample from step h of the kth play.

- $s_{h,k}$: The state, i.e. the board configuration.
- $\pi_{h,k}$: An A-dimension vector such that $\pi_{h,k}(a)$ represents the probability of taking action a in state $s_{h,k}$.
- $z_k \in \{-1, 0, +1\}$: final outcome of the kth play.

Training the Policy Network (Similar to Critic)

Training Samples

K plays of the game and $(s_{h,k}, \pi_{h,k}, z_k)$ is a sample from step h of the kth play.

- $s_{h,k}$: The state, i.e. the board configuration.
- $\pi_{h,k}$: An A-dimension vector such that $\pi_{h,k}(a)$ represents the probability of taking action a in state $s_{h,k}$.
- $z_k \in \{-1, 0, +1\}$: final outcome of the kth play.

Training the policy network

Use mini-batch and loss function

$$L = \sum_b (v_\theta(s_{h,k}) - z_k)^2 - \pi_{h,k} \cdot \log(p_\theta(s_{h,k})) + c\|\theta\|^2$$

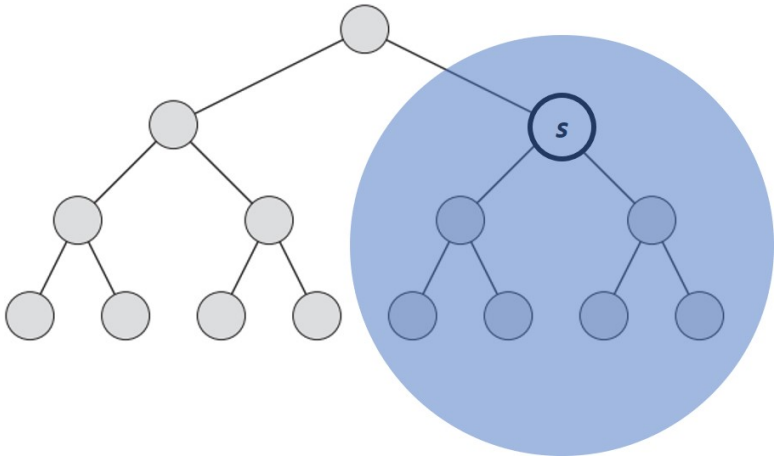
Monte Carlo Tree Search

Policy Improvement with Self-Play (Similar to Actor)

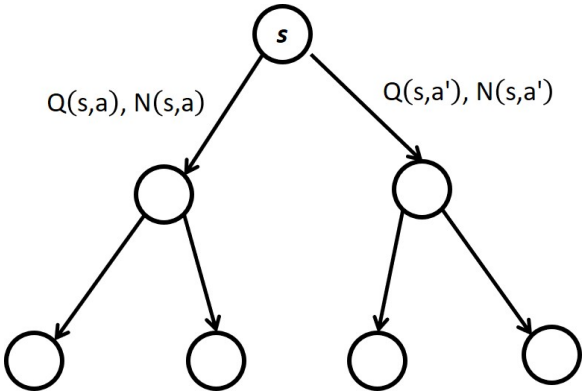
Improve the policy

Given the current board configuration $s_{h,k}$ and policy network θ, generate a new distribution $\pi_{h,k}$ based on Monte Carlo Tree Search (MCTS), i.e. a sample $(s_{h,k}, \pi_{h,k}, ?)$, where ? will be determined after the game ends.

Monte Carlo Tree Search (MCTS)



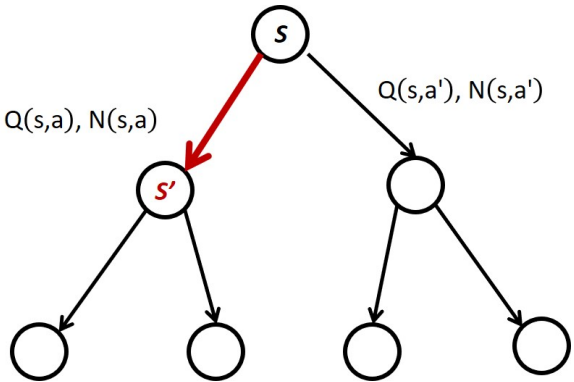
Monte Carlo Tree Search (MCTS)



Each state s maintains the following

- $Q(s, a)$: Q-value for state s and action a.
- $N(s, a)$: number of visits to state s and action a in the self-play.

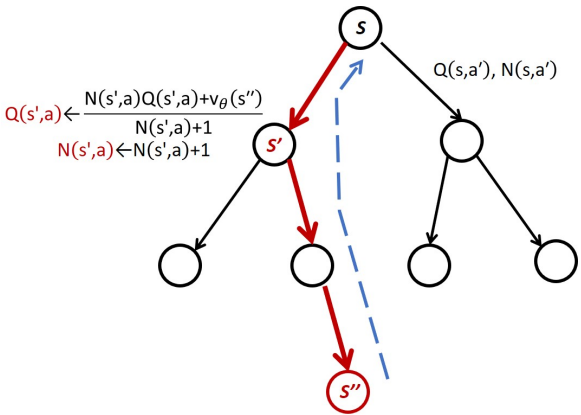
Monte Carlo Tree Search (MCTS): One Simulation



Step 1: calculate

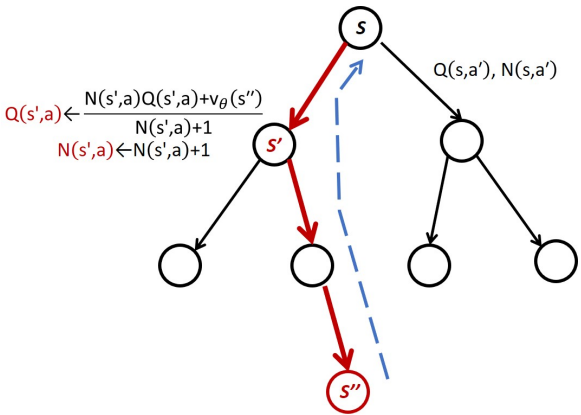
$$a \in \arg\max_u Q(s,u) + hp_\theta(s,u)\frac{\sqrt{\sum_v N(s,v)}}{1+N(s,u)}$$

Monte Carlo Tree Search (MCTS): One Simulation



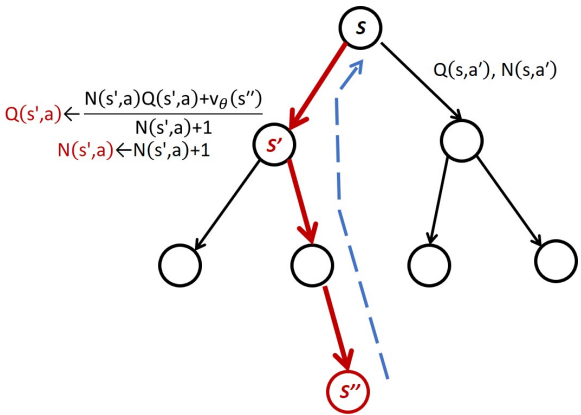
- Step 2: Terminate when meeting a new state or a terminal state and update Q and N on the path.

Monte Carlo Tree Search (MCTS): One Simulation



- Step 3: If s'' is a new state, add it to the tree and set $N(s'', a) = Q(s'', a) = 0$ for all a.

Monte Carlo Tree Search (MCTS): One Simulation



- Step 4: Update $Q(s', a)$ and $N(s', a)$ along the path with $v_\theta(s'')$.

Monte Carlo Tree Search (MCTS): After Many Simulations

Take action

Sample an action according to $\pi(a|s) = \frac{N(s,a)}{\sum_{a'} N(s,a')}$.

Monte Carlo Tree Search (MCTS): After Many Simulations

Take action

Sample an action according to $\pi(a|s) = \frac{N(s,a)}{\sum_{a'} N(s,a')}$.

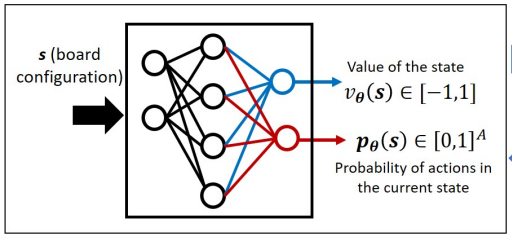
Both black and white are played by the same agent (self-play).

Monte Carlo Tree Search (MCTS): After One Iteration (Many Plays)

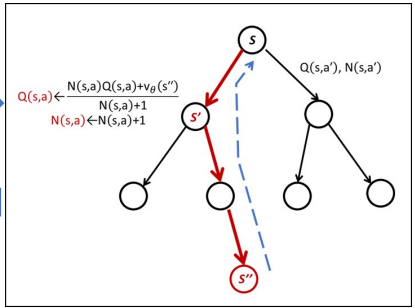
The new and old policies are pit against each other. Update the policy if the new policy wins and otherwise, repeat the iteration to obtain more samples to retrain the policy.

Monte Carlo Tree Search: Example

Overview of AlphaZero

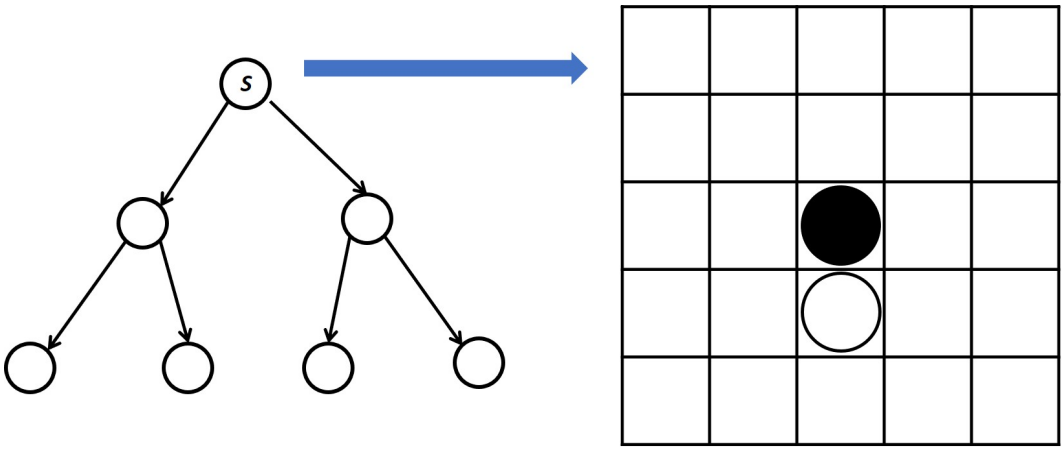


Learn the improved the policy with samples from self-play (similar to Critic)

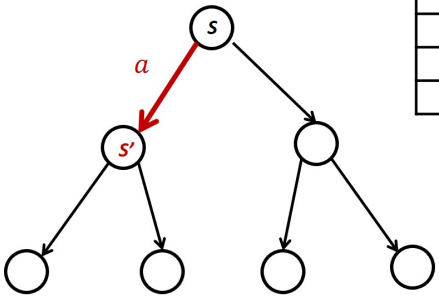


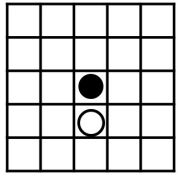
Improve the policy using the policy-network and MCTS (similar to Actor)

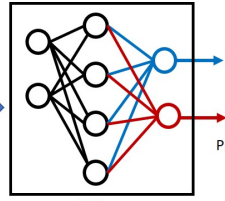
MCTS: Example



MCTS: Example



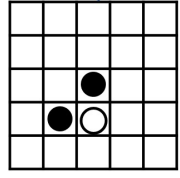




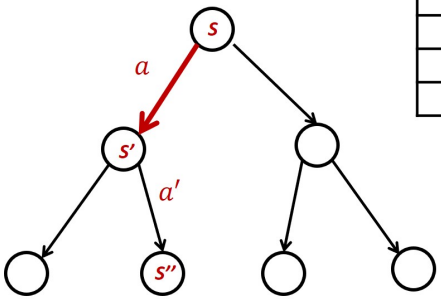
$\boldsymbol{p_\theta(s)} \in [0,1]^A$

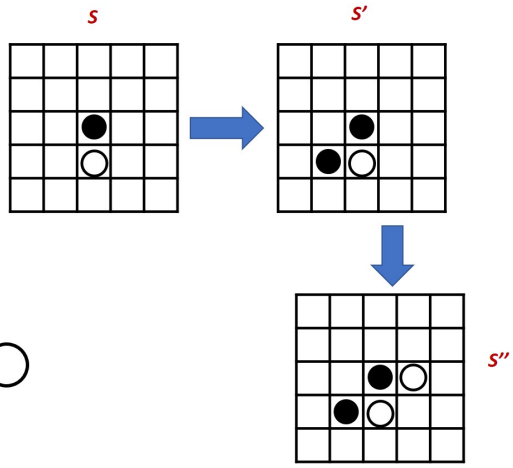
Probability of actions in the current state

$$a \in \arg\max_u Q(s,u) + hp_\theta(s,u)\frac{\sqrt{\sum_v N(s,v)}}{1+N(s,u)}$$

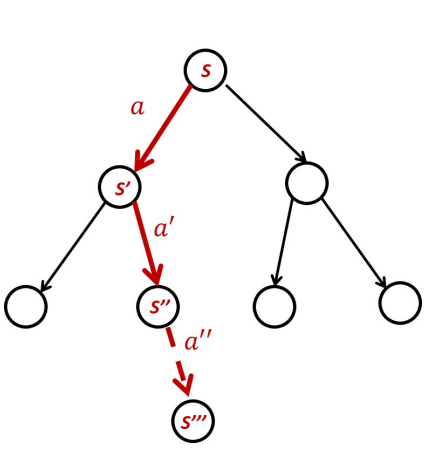


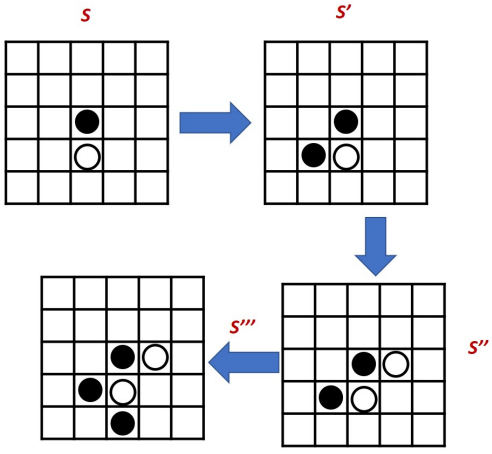
MCTS: Example



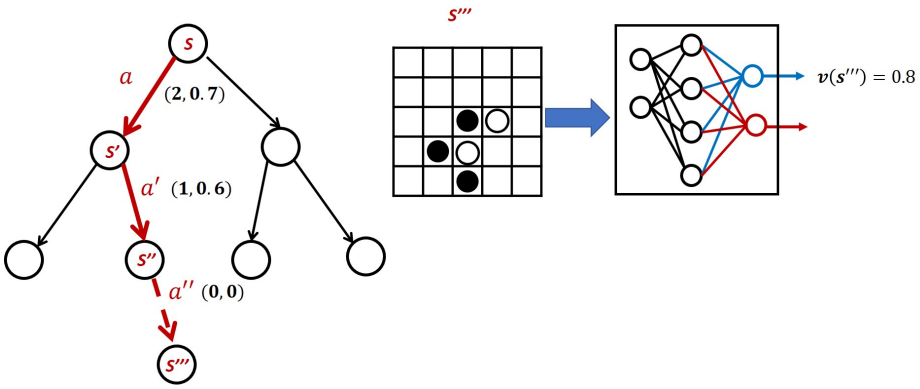


MCTS: Example

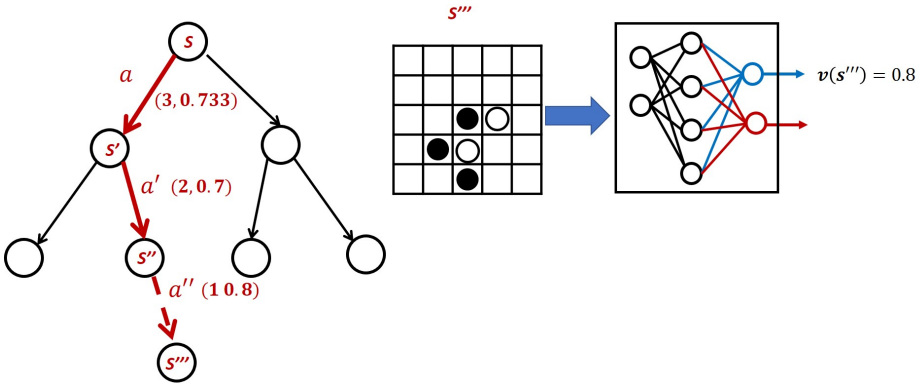




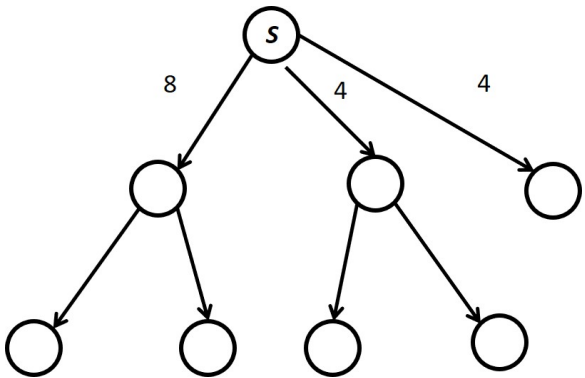
MCTS: Example



MCTS: Example



MCTS: Example



$$\pi(s) = (\frac{8}{16}, \frac{4}{16}, \frac{4}{16})$$
$$= (0.5, 0.25, 0.25)$$

$$\Rightarrow \text{data sample} = (s, (0.5, 0.25, 0.25), ?)$$